

ASSIGNMENT-4

HTNUM-2303A52429

NAME-VALLALA VYSHNAVI

BATCH-32

Question 1: Zero-Shot Prompting (Basic Lab Task)

Task 1:

Write a Python function that classifies a given text as Spam or Not Spam using zero-shot prompting.

Steps:

1. Construct a prompt without any examples.
2. Clearly specify the output labels.
3. Display only the predicted label.

Input: "Congratulations! You have won a free lottery ticket." Expected

Output: Spam

Prompt:

```
#generate a python function that classifies a given text as spam or Not spam based on the presence of certain keywords
```

Code:

```
#generate a python function that classifies a given text as spam or Not spam based on the presence of certain keywords
def classify_text(text):
    """Classify the given text as 'Spam' or 'Not Spam' based on keywords."""
    # Define a set of spam keywords
    spam_keywords = ('win', 'free', 'prize', 'click', 'buy now', 'limited time', 'offer')

    # Convert the text to lowercase for case-insensitive matching
    text_lower = text.lower()

    # Check if any spam keyword is present in the text
    if any(keyword in text_lower for keyword in spam_keywords):
        return "Spam" # Text is classified as Spam
    else:
        return "Not Spam" # Text is classified as Not Spam
```

Output(O/P)

```
Enter the text to classify: "Congratulations! You have won a free lottery ticket."  
Spam
```

Explanation:

This function analyzes the processed text using predefined rules or keywords.

Based on the result, it classifies the text as either "Spam" or "Not Spam"

Question 2: One-Shot Prompting (Emotion detection)

Task 2:

Write a Python program that detects the emotion of a sentence using one-shot prompting.

Emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral'] Steps:

1. Provide one labeled example inside the prompt.
2. Take a sentence as input.
3. Print the predicted emotion

Prompt:

```
'''generate a python program for emotion detection(happy,sad,nervous,angry,excited,neutral)  
example:India won the match -> happy  
The program should take a text input from the user and output the detected emotion.'''
```

Code:

```
'''generate a python program for emotion detection(happy,sad,nervous,angry,excited,neutral)
example:India won the match -> happy
The program should take a text input from the user and output the detected emotion.'''
def detect_emotion(text):
    text = text.lower()
    if any(word in text for word in ['won', 'great', 'fantastic', 'amazing', 'joyful', 'happy', 'excited']):
        return 'happy'
    elif any(word in text for word in ['lost', 'sad', 'unfortunate', 'depressed', 'down']):
        return 'sad'
    elif any(word in text for word in ['nervous', 'anxious', 'worried', 'tense']):
        return 'nervous'
    elif any(word in text for word in ['angry', 'furious', 'irate', 'mad']):
        return 'angry'
    elif any(word in text for word in ['excited', 'thrilled', 'eager']):
        return 'excited'
    else:
        return 'neutral'
if __name__ == "__main__":
    user_input = input("Enter a text: ")
    emotion = detect_emotion(user_input)
    print(f"The detected emotion is: {emotion}")
```

Output:

```
PS C:\Users\Admin> & C:/Users/Admin/Ap
Enter a text: India won the match
The detected emotion is: happy
PS C:\Users\Admin> █
```

```
Enter a text: hi im vyshnavi 2429
The detected emotion is: neutral
PS C:\Users\Admin> █
```

Explanation:

This program defines a function `detect\_emotion` that analyzes the input text for specific keywords associated with different emotions.

It checks for the presence of these keywords and returns the corresponding emotion. If no keywords are found, it defaults to 'neutral'.

Question 3: Few-Shot Prompting (Student Grading Based on Marks)

Task 3:

Write a Python program that predicts a student's grade based on marks using few-shot prompting.

Grades: ['A', 'B', 'C', 'D', 'F']

Grading Criteria (to be inferred from examples):

- 90–100 → A
- 80–89 → B
- 70–79 → C
- 60–69 → D
- Below 60 → F

Prompt:

```
users > Admin > Downloads > AI ASSISTANCE > 2041.py > ...  
'''generate a python program for calculating the grades of students  
grades:[a,b,c,d,f]  
90-100->a  
80-89->b  
70-79->c  
60-69->d  
below 60->f  
...  
'''
```

Code:

```
60-69->d  
below 60->f  
...  
  
def calculate_grade(score):  
    if 90 <= score <= 100:  
        return 'A'  
    elif 80 <= score < 90:  
        return 'B'  
    elif 70 <= score < 80:  
        return 'C'  
    elif 60 <= score < 70:  
        return 'D'  
    elif 0 <= score < 60:  
        return 'F'  
    else:  
        return 'Invalid score'  
  
def main():  
    try:  
        score = float(input("Enter the student's score (0-100): "))  
        grade = calculate_grade(score)  
        print(f"The student's grade is: {grade}")  
    except ValueError:  
        print("Please enter a valid number.")  
  
if __name__ == "__main__":  
    main()
```

Output:

```
Enter the student's score (0-100): 99
The student's grade is: A
```

```
Enter the student's score (0-100): 150
The student's grade is: Invalid score
```

Explanation:

The program defines a function `calculate\_grade` that takes a numerical score as input and returns the corresponding letter grade based on predefined ranges.

It provides a clear and structured approach to handling user input and output, making it easy to use and understand.

Question 4: Multi-Shot Prompting (Indian Zodiac Sign Prediction using Month Name)

Task 4:

Write a Python program that predicts a person's Indian Zodiac sign (Rashi) based on the month of birth (month name) using multi-shot prompting.

Indian Zodiac Order (Simplified Month-Based Model):

The Indian Zodiac cycle starts in March with Mesha and follows this order:

March → Mesha

April → Vrishabha

May → Mithuna

June → Karka

July → Simha

August → Kanya

September → Tula

October → Vrischika

November → Dhanu

December → Makara

January → Kumbha

February → Meena

Prompt:

```
...  
generate a python program that predicts a persons Indian Zodiac sign based on the month of birth  
March → Mesha  
April → Vrishabha  
May → Mithuna  
June → Karka  
July → Simha  
August → Kanya  
September → Tula  
October → Vrischika  
November → Dhanu  
December → Makara  
January → Kumbha  
February → Meena  
take input from the user  
...
```

Code:

```
def get_indian_zodiac_sign(month):
    zodiac_signs = {
        "March": "Mesha",
        "April": "Vrishabha",
        "May": "Mithuna",
        "June": "Karka",
        "July": "Simha",
        "August": "Kanya",
        "September": "Tula",
        "October": "Vrischika",
        "November": "Dhanu",
        "December": "Makara",
        "January": "Kumbha",
        "February": "Meena"
    }

    return zodiac_signs.get(month, "Invalid month")

def main():
    month = input("Enter your month of birth (e.g., March, April, etc.): ")
    zodiac_sign = get_indian_zodiac_sign(month)
    print(f"Your Indian Zodiac sign is: {zodiac_sign}")

if __name__ == "__main__":
    main()
```

Output:

```
Enter your month of birth (e.g., March, April, etc.): December
Your Indian Zodiac sign is: Makara
PS C:\Users\Admin>
```

Explanation:

AI was helpful in quickly generating a structured program that maps months to Indian Zodiac signs, saving time and effort in coding from scratch.

The code defines a function to retrieve the Zodiac sign based on user input and uses a dictionary for efficient look-up, ensuring clarity and ease of maintenance.

Question 5: Result Analysis Based on Marks

Task 5:

Write a Python program that determines whether a student Passes or Fails based on marks using Chain-of-Thought (CoT) prompting.

Result Categories: ['Pass', 'Fail']

Prompt:

```
'''Read marks from the user
validate the range of marks (0-100) if the marks are out of range print "Invalid marks"
if the marks are above 35 print "Pass" else print "Fail"
display the result
'''
```

Code:

```
marks = float(input("Enter the marks (0-100): "))
if marks < 0 or marks > 100:
    print("Invalid marks")
elif marks > 35:
    print("Pass")
else:
    print("Fail")
```

Output:

```
PS C:\Users\Admin> & C:\Users\Admin>
Enter the marks (0-100): 95
Pass
```


Explanation:

The code reads a float input for marks, checks if it's within the valid range (0-100), and then determines if the student has passed or failed based on the marks.

Question 6: Voting Eligibility Check (Chain-of-Thought Prompting)

Task 6:

Write a Python program that determines whether a person is eligible to vote using Chain-of-Thought (CoT) prompting.

Prompt:

```
'''Read age from the user
validate the range of age above or equal to 18 if the age is out of range print "Invalid age"
if the age is equal or above 18 print "Eligible" else print "Not eligible "
display the result
'''
```

Code:

```
age = int(input("Enter your age: "))
if age < 0:
    print("Invalid age")
elif age >= 18:
    print("Eligible")
else:
    print("Not eligible")
```

Output:

```
Enter your age: 5
Not eligible
PS C:\Users\Admin>
```

Explanation:

The code prompts the user to input their age and converts it to an integer.

It then checks if the age is less than 0, in which case it prints "Invalid age". If the age is 18 or older, it prints "Eligible".

Question 7: Prompt Chaining (String Processing – Palindrome Names)

Task 7:

Write a Python program that uses the prompt chaining technique to identify palindrome names from a list of student names.

Prompt:

```
'''
generate a list with student names
traverse the list and generate the reverse of each name
if the names are same print "Palindrome" else print "Not a Palindrome"
display the result
'''
```

Code:

```
student_names = ["Anna", "Bob", "Civic", "David", "Eve"]
for name in student_names:
    reversed_name = name[::-1]
    if name.lower() == reversed_name.lower():
        print(f"{name}: Palindrome")
    else:
        print(f"{name}: Not a Palindrome")
```

Output:

```
Anna: Palindrome
Bob: Palindrome
Civic: Palindrome
David: Not a Palindrome
Eve: Palindrome
PS C:\Users\Admin> 
```

Explanation:

The code defines a list of student names and iterates through each name.

For each name, it generates the reverse of the name using slicing.

It then checks if the original name (converted to lowercase) is the same as the reversed name (also converted to lowercase).

If they are the same, it prints "Palindrome"; otherwise, it prints "Not a Palindrome".

Question 8: Prompt Chaining (String Processing – Word Length Analysis)

Task 8:

Write a Python program that uses prompt chaining to analyze a list of words. In the first prompt, generate a list of words. In the second prompt, traverse the list and calculate the length of each word. In the third prompt, use the output of the previous step to determine whether each word is Short (length less than 5) or Long (length greater than or equal to 5), and display the result for each word

Prompt:

```
'''
generate a list of words
traverse the list and calculate the length of each word
store the lengths in a new list
if the length of word is greater than or equal to 5 print long else short
display the result of the word
'''
```

Code:

```
words = ["apple", "bat", "banana", "cat", "elephant", "dog", "grape"]
lengths = []
for word in words:
    length = len(word)
    lengths.append(length)
    if length >= 5:
        print(f"{word}: long")
    else:
        print(f"{word}: short")
print("Lengths of words:", lengths)
```

Output:

```
apple: long
bat: short
banana: long
cat: short
elephant: long
dog: short
grape: long
Lengths of words: [5, 3, 6, 3, 8, 3, 5]
```

Explanation:

This code generates a list of words and iterates through each word to calculate its length.

It stores the lengths in a new list called 'lengths'.

For each word, it checks if the length is greater than or equal to 5 and prints "long" or "short" accordingly.

Finally, it displays the list of lengths of the words.